

CS-600: Software Design and Development

6-2 Activity: Feature Implementation and Integration Testing

Malgorzata Debska

Southern New Hampshire University

Matthew Scuteri

May 28, 2026

Email Feature Implementation and Integration Testing

For this activity, I implemented a new email feature in the existing contact manager application. The original application displayed contact information on a table, including first name, middle name, last name, and suffix. The new requirement was to add an Email column and automatically generate an email address for each contact using the required format of firstname.lastname@example.com. My goal was to add this feature without changing the existing database structure or disrupting the current login and name list functionality.

Feature Implementation

The first step in my implementation process was reviewing the existing project structure to understand where the contact data was stored and where the table was created in the user interface. I showed the Name entity as the best place to add the email generation logic because the email address depends on the first name and last name already stored in each record. Instead of adding a new database field, I created a method that dynamically generates the email address from the existing firstName and lastName values. This approach kept the update simple and avoided unnecessary database changes. I also updated the Vaadin grid view so that the generated email address would appear as a new column in the Name List table.

The implementation meets the identified requirement because each record now displays an email address in the correct format. For example, contact the first name Tayla and last name Bannister displays as tayla.bannister@example.com. This confirms that the feature uses the required naming pattern and applies to it consistently across the contact list. The Email column also

appears beside the existing fields, which shows that the new feature was integrated into the current table instead of replacing or disrupting existing information.

Testing Process and Results

After implementing the feature, I tested the application in the Codio environment. I first ran the application using the Maven wrapper command. The terminal showed that Tomcat started successfully on port 8443 and that the application started without errors. This confirmed that the code was compiled and that the application could still launch after the new feature was added.

I then opened the application in the browser and logged in through the existing password-protected login page. The login process worked correctly, which showed that the authentication functionality was not affected by my changes.

Once I logged in, I opened the Name List page and reviewed the table. The existing columns for First Name, Middle Name, Last Name, and Suffix were still visible, and the new Email column appeared correctly. I checked several records in the table and confirmed that each email address was generated using the contact's first and last name. The email values matched the required format and displayed consistently for multiple records. This testing proved that the new feature worked as expected and that it was successfully integrated with the existing data display.

Integration Testing Analysis

The results of the integration testing were successful. The application started normally, the login page loaded correctly, the user was able to access the Name List page, and the table displayed

both the original contact data and the new Email column. No errors appeared in the browser or terminal during testing.

The existing functionality continued to work, which is important because a new feature should improve the application without breaking what already exists. Since the email address is generated dynamically from existing fields, the feature also avoids adding extra complexity to the database or requiring added stored data.

Stability and Reliability

Testing confirmed the stability and reliability of the updated application because the feature worked across the full application flow. I tested the application from startup, through login, and finally through the contact list display. This process showed that the new code did not interfere with system startup, security, database retrieval, or user interface rendering.

The application remained stable while displaying the new email data, and the generated values were predictable and consistent. No crashes, login issues, or display errors occurred during testing. These results prove that the application stays reliable after the feature is added.

This activity showed the importance of connecting implementation decisions to requirements and verifying those decisions through testing. The new Email column supports the system's goal of making the contact manager more useful by providing an added contact detail for each record. At the same time, the implementation was kept simple, maintainable, and easy to understand.

Generating the email address from existing name data, the application now provides the required

information without unnecessary database changes. The successful test results confirm that the updated application meets the feature requirements and continues to work reliably.



